



Algoritmo genético para TSP (Travelling Salesman Problem)

Programación Paralela y Distribuida

Manuel Candil Baeza, Pablo Pérez Santiago, Alejandro Ramos Rodríguez, Alejandro Sanz Huerta

CURSO 2025/26

Índice general

1. Análisis	5
1.1. Explicación del problema	5
1.2. Aplicación de la neurona	6
1.2.1. Integración de la neurona en el algoritmo genético	6
1.2.2. Control adaptativo del operador de mutación	6
1.2.3. Evolución temporal y memoria del sistema	7
1.2.4. Aplicación uniforme en las distintas versiones del algoritmo	7
1.2.5. Ventajas del enfoque propuesto	7
1.3. Elección de los parámetros de Izhikevich	8
1.3.1. Parámetros del modelo	8
1.3.2. Uso de patrones de disparo predefinidos	8
1.3.3. Justificación de la diversidad de parámetros	9
2. Diseño	11
2.1. Diagrama de flujo del programa	11
2.2. Estructura de la neurona	16
2.2.1. Estructura de datos Neurona	16
2.2.2. Inicialización de la neurona	16
2.2.3. Funciones observadoras	17
2.2.4. Actualización del estado neuronal	17
3. Resultados	19
3.1. Definición de las pruebas y metodología	19
3.1.1. Medición de tiempos	19
3.1.2. Parámetros experimentales	20
3.1.3. Estructura de los datos obtenidos	20
3.2. Entorno de pruebas (Hardware y Software)	20
3.3. Análisis de Speedup y Eficiencia Paralela	21
3.3.1. Interpretación de las métricas de rendimiento	21
3.3.2. Análisis de la precisión (Error Promedio)	23

3.4. Conclusiones 24

Capítulo 1

Análisis

1.1. Explicación del problema

El objetivo principal de este trabajo es evaluar y comparar el rendimiento de distintos operadores dentro de un algoritmo genético, considerando la influencia de la dinámica de neuronas de tipo Izhikevich en el proceso de mutación. Para ello se plantea un problema de ejemplo bien conocido en computación y optimización combinatoria: el *Travelling Salesman Problem* (TSP).

En este contexto, el TSP consiste en encontrar el camino más corto que recorra un conjunto de ciudades exactamente una vez y regrese al punto de partida. Para modelar este problema dentro del algoritmo genético, se utiliza una representación basada en caminos hamiltonianos, en la que cada individuo de la población corresponde a una permutación de los nodos del grafo que representa las ciudades y sus conexiones.

El enfoque adoptado permite estudiar de manera sistemática cómo la inclusión de una neurona Izhikevich que controla la probabilidad de mutación afecta a la calidad de la solución y al tiempo de convergencia del algoritmo. Además, la neurona Izhikevich introduce un componente dinámico y adaptativo, lo que permite evaluar si la utilización de patrones de disparo neuronal mejora la exploración del espacio de soluciones frente a un algoritmo genético convencional.

De esta manera, el problema sirve como un banco de pruebas para comparar:

- La influencia de la neurona en la dinámica de mutación y en la convergencia del algoritmo.

- La eficiencia computacional de versiones secuenciales y paralelas del algoritmo.

En resumen, el TSP con representación de caminos hamiltonianos proporciona un escenario controlado y ampliamente estudiado que permite analizar de forma rigurosa el impacto de incorporar una neurona pulsante en un algoritmo genético.

1.2. Aplicación de la neurona

Con el objetivo de mejorar el comportamiento del algoritmo genético aplicado al problema del TSP, se ha incorporado un modelo de neurona como mecanismo de control del operador de mutación. En lugar de emplear una probabilidad de mutación fija, se utiliza el comportamiento de una neurona de tipo *Izhikevich* cuyo estado interno evoluciona a lo largo de la ejecución del algoritmo y condiciona dinámicamente la aplicación de dicho operador.

Esta aproximación permite introducir un comportamiento adaptativo en el proceso evolutivo, dotando al algoritmo de una capacidad de respuesta dependiente de su propia evolución temporal, sin necesidad de ajustar manualmente parámetros estáticos.

1.2.1 Integración de la neurona en el algoritmo genético

La neurona se integra directamente dentro del bucle principal del algoritmo genético, actuando como elemento que influye en la generación de nuevos individuos. Para cada intento de creación de un nuevo individuo, el algoritmo sigue las fases clásicas de selección, cruce y mutación, incorporando la neurona como el elemento decisivo en esta última etapa.

La neurona se inicializa al comienzo de la ejecución del algoritmo y su estado se mantiene y actualiza de forma continua durante todas las generaciones, introduciendo así una dependencia temporal entre iteraciones consecutivas.

1.2.2 Control adaptativo del operador de mutación

El papel principal de la neurona es regular la aplicación del operador de mutación. En cada iteración del algoritmo comprueba el valor de una magnitud derivada del estado interno de la neurona si este supera el umbral establecido en 0, entonces se aplica el operador de mutación al individuo recién generado.

De este modo, la probabilidad de mutación deja de ser constante y pasa a depender del estado dinámico de la neurona, el cual evoluciona en función de la actividad previa del sistema.

1.2.3 Evolución temporal y memoria del sistema

Una de las principales aportaciones de esta integración es la introducción de una forma de memoria temporal en el algoritmo genético. El estado de la neurona no se reinicia entre generaciones, sino que se actualiza tras cada intento de creación de un nuevo individuo, independientemente de si se ha aplicado mutación o no.

Este comportamiento provoca que la probabilidad de mutación esté influenciada por la historia reciente del algoritmo, permitiendo alternar de forma natural entre fases de mayor exploración del espacio de soluciones y fases de explotación de soluciones prometedoras.

1.2.4 Aplicación uniforme en las distintas versiones del algoritmo

La neurona se aplica de forma coherente tanto en las versiones secuenciales como en las versiones paralelas del algoritmo genético. En todos los casos, su función es actuar como regulador adaptativo del operador de mutación, sin modificar el resto de componentes del algoritmo.

Esto permite analizar de forma aislada el impacto de la neurona sobre la calidad de las soluciones obtenidas, independientemente del modelo de ejecución empleado.

1.2.5 Ventajas del enfoque propuesto

La incorporación de una neurona de tipo *Izhikevich* como elemento de control presenta varias ventajas relevantes:

- Introduce adaptación dinámica basada en la evolución del propio algoritmo.
- Mantiene un coste computacional reducido.
- Facilita la experimentación con distintos comportamientos dinámicos sin alterar la estructura del algoritmo genético.

1.3. Elección de los parámetros de Izhikevich

La elección de los parámetros del modelo de neurona de Izhikevich constituye un aspecto clave para el correcto funcionamiento del mecanismo de control del operador de mutación. Dado que el objetivo principal de la neurona no es simular con exactitud un sistema biológico real, sino proporcionar un comportamiento dinámico y adaptable al algoritmo genético, los parámetros se seleccionan atendiendo a criterios funcionales y experimentales.

El modelo de Izhikevich permite ajustar distintos patrones de disparo mediante cuatro parámetros fundamentales: a , b , c y d . Estos parámetros controlan la dinámica interna de la neurona y, por tanto, la evolución de la señal utilizada para regular la mutación.

1.3.1 Parámetros del modelo

El modelo de neurona de Izhikevich se caracteriza por los siguientes parámetros:

- a : controla la velocidad de recuperación de la variable interna de la neurona.
- b : determina la sensibilidad de dicha recuperación respecto al potencial de membrana.
- c : valor al que se reinicia el potencial de membrana tras un disparo.
- d : incremento aplicado a la variable de recuperación tras un disparo.

La combinación de estos parámetros define el patrón de disparo de la neurona y, en consecuencia, la evolución temporal de la probabilidad de mutación aplicada en el algoritmo genético.

1.3.2 Uso de patrones de disparo predefinidos

En lugar de ajustar manualmente valores continuos para los parámetros, se han empleado conjuntos de parámetros asociados a patrones de disparo bien conocidos en la literatura del modelo de Izhikevich. Estos patrones representan distintos comportamientos dinámicos de activación neuronal, como disparo regular, ráfagas o activación rápida.

Este enfoque permite:

- Simplificar el proceso de configuración del algoritmo.

- Facilitar la comparación entre distintos comportamientos dinámicos.
- Evaluar el impacto de la dinámica neuronal sobre el rendimiento del algoritmo genético.

Cada conjunto de parámetros define un tipo de neurona distinto, que se mantiene constante durante toda la ejecución del algoritmo. Cada conjunto de parámetros define un tipo de neurona distinto, que se mantiene constante durante toda la ejecución del algoritmo. Los patrones de disparo empleados corresponden a configuraciones ampliamente utilizadas en la literatura del modelo de Izhikevich, descritas en el trabajo original del autor [2].

1.3.3 Justificación de la diversidad de parámetros

La utilización de varios conjuntos de parámetros responde a la necesidad de analizar cómo diferentes dinámicas neuronales influyen en el proceso evolutivo. Algunos patrones generan activaciones más frecuentes, lo que se traduce en mayores tasas de mutación, mientras que otros producen activaciones más espaciadas, favoreciendo una evolución más conservadora.

Esta diversidad permite estudiar el comportamiento del algoritmo bajo distintas estrategias implícitas de exploración y explotación, sin modificar la estructura del algoritmo genético ni introducir nuevos operadores.

Capítulo 2

Diseño

2.1. Diagrama de flujo del programa

Algoritmo Secuencial

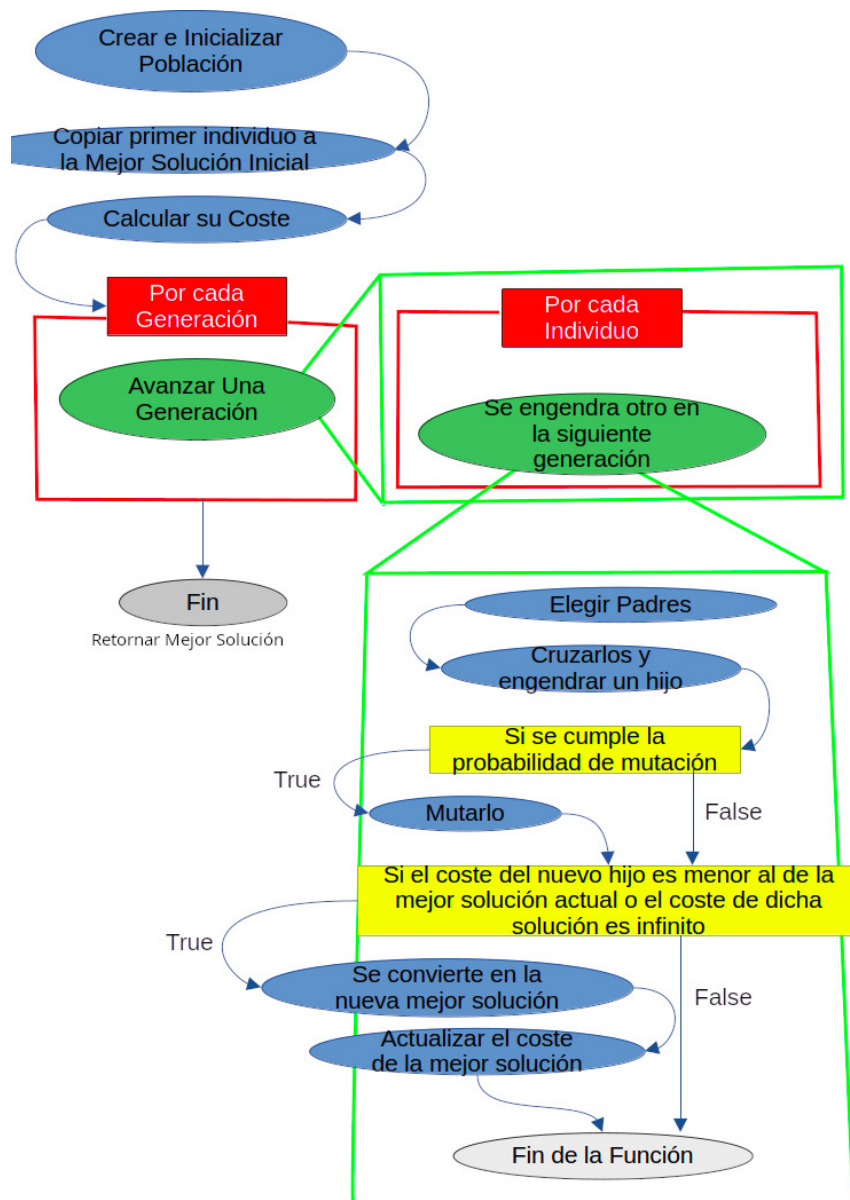


Figura 2.1: Diagrama de flujo de la versión Secuencial.

1. Inicialización

Se genera una población inicial de individuos (posibles rutas) de forma aleatoria.

2. Evaluación Inicial

Se establece el primer individuo como la *mejor solución actual*. Se calcula su coste para tener una referencia base en las comparaciones posteriores.

3. Evolución por Generaciones

Se procesa cada generación de forma iterativa, creando una nueva población a partir de la anterior mediante los siguientes pasos:

- **Selección:** Se eligen los padres de forma aleatoria para la reproducción.
- **Cruce:** Se combinan los genes de los padres para generar descendencia.
- **Mutación:** Se aplica una alteración aleatoria al nuevo individuo si se cumple la probabilidad de mutación definida por el modelo.
- **Evaluación y Actualización:** Se calcula el coste del nuevo individuo. Si este es menor al mejor coste registrado (o si el actual es infinito debido a que las soluciones iniciales en el TSP suelen ser rutas inviables), se actualiza la mejor solución con este individuo.
- **Inserción:** El nuevo individuo se añade a la población de la siguiente generación.

4. Finalización

Una vez completadas todas las generaciones, el algoritmo retorna la mejor solución hallada.

Algoritmo Paralelo

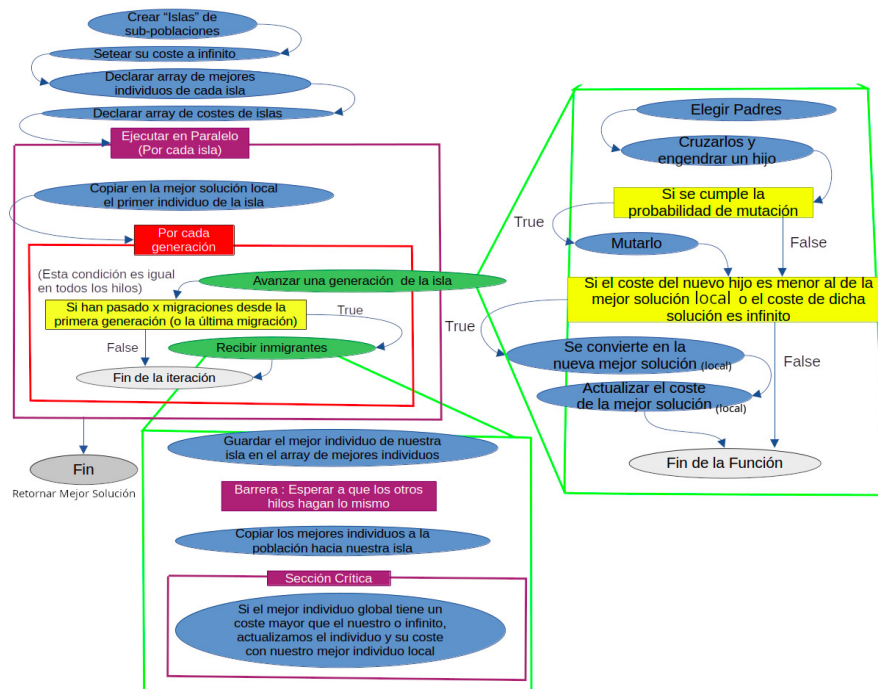


Figura 2.2: Diagrama de flujo de la versión Paralela.

Este enfoque emplea un modelo de islas, dividiendo la población total en sub-poblaciones que se procesan de forma simultánea en hilos independientes. Para evitar caer en mínimos locales, se implementa un mecanismo de migración periódica de los mejores individuos entre islas.

1. Inicialización Distribuida

Cada isla inicializa su propia sub-población de forma independiente. Al igual que en el modelo secuencial, cada hilo establece su mejor solución local inicial (generalmente con un coste infinito).

2. Estructuras de Control

Los mejores individuos y sus respectivos costes se almacenan en estructuras de datos globales (arrays) cuya longitud coincide con el número de hilos o islas activos.

3. Ejecución Paralela y Sincronización

- Cada isla evoluciona de forma autónoma durante un número n de generaciones siguiendo el esquema secuencial.
- Tras completar su ciclo local, cada hilo registra su mejor individuo en la posición correspondiente del array global.
- Se establece una barrera de sincronización para asegurar que todos los hilos hayan terminado su fase de computación antes de proceder.

4. Fase de Migración

Los hilos copian el conjunto de las mejores soluciones globales a sus respectivas poblaciones locales, enriqueciendo la diversidad genética de cada isla con los éxitos de las demás.

5. Actualización del Óptimo Global

Mediante una sección crítica (para garantizar la exclusión mutua), cada hilo compara su mejor resultado con el récord global. Si el coste local es menor, se actualiza la solución maestra del algoritmo.

6. Finalización

Tras completar todos los ciclos de migración y evolución, se retorna el mejor individuo global encontrado por el conjunto de las islas.

2.2. Estructura de la neurona

La neurona utilizada se ha implementado mediante una estructura de datos y un conjunto de funciones que encapsulan tanto su estado interno como su dinámica temporal, simulando el comportamiento de una clase al estar limitados por el lenguaje C. Esta implementación permite una integración clara de la neurona dentro del algoritmo genético y facilita la reutilización de distintos comportamientos neuronales.

2.2.1 Estructura de datos Neurona

La estructura `Neurona` representa una neurona individual e incluye tanto los parámetros del modelo como sus variables dinámicas. En ella se almacenan los cuatro parámetros característicos del modelo de Izhikevich (a , b , c y d), así como las variables de estado v (potencial de membrana) y u (variable de recuperación). Además, se incorpora la variable I , que representa la corriente de entrada aplicada a la neurona en cada instante de simulación.

Esta separación entre parámetros fijos y estado dinámico permite mantener constante el tipo de neurona durante la ejecución del algoritmo, mientras que su actividad evoluciona en el tiempo.

2.2.2 Inicialización de la neurona

La función `crear_neurona` se encarga de inicializar una neurona genérica a partir de los parámetros del modelo de Izhikevich. Esta función recibe los valores de los cuatro parámetros característicos (a , b , c y d) y asigna las variables dinámicas v , u y I a cero, proporcionando un estado inicial neutro para la simulación.

A partir de esta función base se implementan varias funciones específicas de inicialización, cada una asociada a un patrón de disparo concreto:

- `crear_neurona_RS()`: Regular Spiking, con parámetros $a = 0,02$, $b = 0,2$, $c = -65,0$, $d = 8,0$.
- `crear_neurona_IB()`: Intrinsically Bursting, con parámetros $a = 0,02$, $b = 0,2$, $c = -55,0$, $d = 4,0$.
- `crear_neurona_CH()`: Chattering, con parámetros $a = 0,02$, $b = 0,2$, $c = -50,0$, $d = 2,0$.

- `crear_neurona_FS()`: Fast Spiking, con parámetros $a = 0,1$, $b = 0,2$, $c = -65,0$, $d = 2,0$.
- `crear_neurona_TC1()`: Thalamo-cortical 1, con parámetros $a = 0,02$, $b = 0,25$, $c = -65,0$, $d = 0,05$.
- `crear_neurona_TC2()`: Thalamo-cortical 2, con parámetros $a = 0,1$, $b = 0,25$, $c = -65,0$, $d = 2,0$.
- `crear_neurona_RZ()`: Resonator, con parámetros $a = 0,1$, $b = 0,26$, $c = -65,0$, $d = 2,0$.
- `crear_neurona_LTS()`: Low-Threshold Spiking, con parámetros $a = 0,02$, $b = 0,25$, $c = -65,0$, $d = 2,0$.

Estas funciones permiten inicializar de forma directa distintos tipos de neuronas sin necesidad de ajustar manualmente los parámetros, simplificando la experimentación con distintos patrones de disparo y facilitando la integración de la neurona dentro del algoritmo genético.

2.2.3 Funciones observadoras

Para acceder de forma controlada a los parámetros y al estado interno de la neurona, se han definido varias funciones observadoras: `neurona_get_a`, `neurona_get_b`, `neurona_get_c`, `neurona_get_d`, `neurona_get_v`, `neurona_get_u` y `neurona_get_I`. Estas funciones permiten consultar el valor de cada parámetro o variable de estado sin modificar directamente la estructura, favoreciendo una mayor claridad y seguridad en el uso de la neurona dentro del programa.

2.2.4 Actualización del estado neuronal

La dinámica temporal de la neurona se implementa en la función `spike_neurona`. Esta función calcula el siguiente estado de la neurona a partir de las ecuaciones diferenciales del modelo de Izhikevich, utilizando el método de Euler para su integración numérica. En cada llamada, se actualizan las variables v y u en función de su estado previo y de la corriente de entrada I .

Asimismo, `spike_neurona` incorpora la lógica de detección de disparos: cuando el potencial de membrana alcanza el umbral definido, la neurona se reinicia al valor c y la variable de recuperación se incrementa en d . Este mecanismo reproduce el comportamiento pulsante característico del modelo y permite simular distintos patrones de disparo de forma eficiente.

Capítulo 3

Resultados

3.1. Definición de las pruebas y metodología

Con el objetivo de realizar una comparativa de rendimiento equitativa, se han sometido todas las variantes del algoritmo a un mismo conjunto de pruebas estandarizadas. Las variantes analizadas son las siguientes:

- **Versión secuencial base:** Algoritmo estándar sin implementación de modelos neuronales.
- **Versión paralela base:** Algoritmo paralelizado sin implementación de modelos neuronales.
- **Versiones con modelo neuronal (Secuencial y Paralela):** Implementación que integra una neurona “genérica”, permitiendo la configuración dinámica de parámetros. Considerando los distintos tipos de neurona (según el modelo de Izhikevich), se obtienen un total de 8 variantes adicionales para el análisis.

3.1.1 Medición de tiempos

Para la medición del tiempo de ejecución se han empleado las funciones nativas de la biblioteca **OpenMP** (`omp_get_wtime`). La metodología empleada corresponde a la **Medida Indirecta**, realizándose 30 repeticiones por cada prueba para garantizar la significancia estadística de los resultados.

3.1.2 Parámetros experimentales

Para evaluar la escalabilidad, los algoritmos se sometieron a pruebas de estrés incrementando dos parámetros fundamentales: el tamaño de la población y el número de generaciones.

Tras descartar pruebas preliminares con cargas de trabajo reducidas (1000 generaciones y 50 individuos), se estableció el siguiente diseño experimental para evaluar el rendimiento con grandes volúmenes de datos:

- **Valor de tamaño de población:** 500.
- **Valor de número de generaciones:** 20000.

3.1.3 Estructura de los datos obtenidos

Los resultados obtenidos (datos crudos) se almacenaron en archivos de texto plano. Cada archivo corresponde a una prueba específica bajo los parámetros del modelo de Izhikevich, siguiendo esta estructura de columnas:

- **Número de hilos:** Cantidad de hilos de ejecución empleados (el valor 0 denota la versión secuencial).
- **Tiempo de ejecución:** Medido en segundos.
- **Error absoluto promedio:** Métrica de precisión del resultado.

3.2. Entorno de pruebas (Hardware y Software)

Las pruebas se ejecutaron en un ordenador portátil bajo el Subsistema de Windows para Linux (WSL 2). Debido a la virtualización de recursos que implica WSL, las características efectivas del entorno de ejecución fueron:

- **Sistema Operativo:** Debian 12 (ejecutándose sobre WSL).
- **Procesador:** Intel Core i5-12450H. Para más detalles técnicos, consultar [1].
- **Memoria RAM:** 8 GB DDR4 a 4800 MT/s (Memoria asignada al subsistema).

3.3. Análisis de Speedup y Eficiencia Paralela

En esta sección se presentan los resultados obtenidos tras la ejecución de las pruebas bajo el “Caso 5” (parámetros de máxima carga). A continuación, se detallan las métricas de eficiencia paralela, aceleración (SpeedUp) y precisión de los modelos.

3.3.1 Interpretación de las métricas de rendimiento

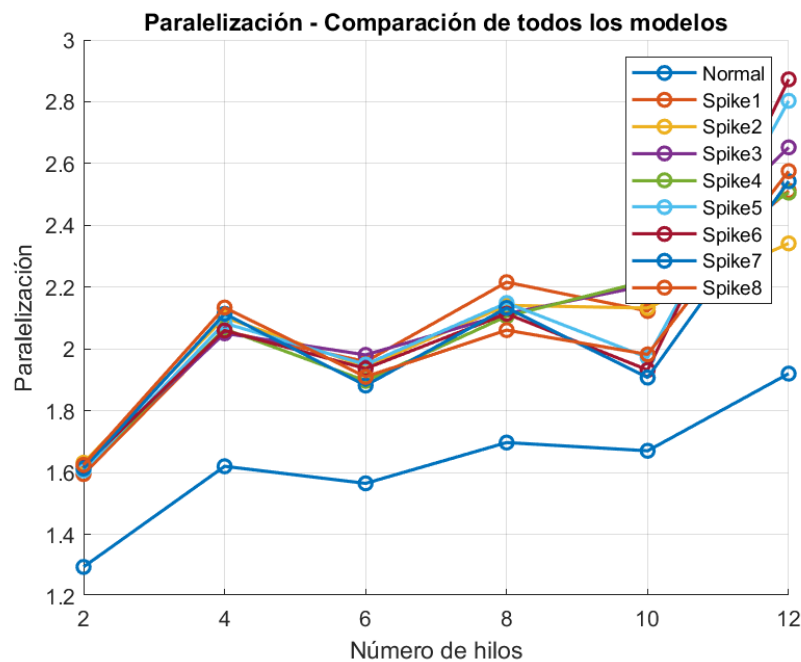


Figura 3.1: Evolución de la eficiencia paralela en función del número de hilos.

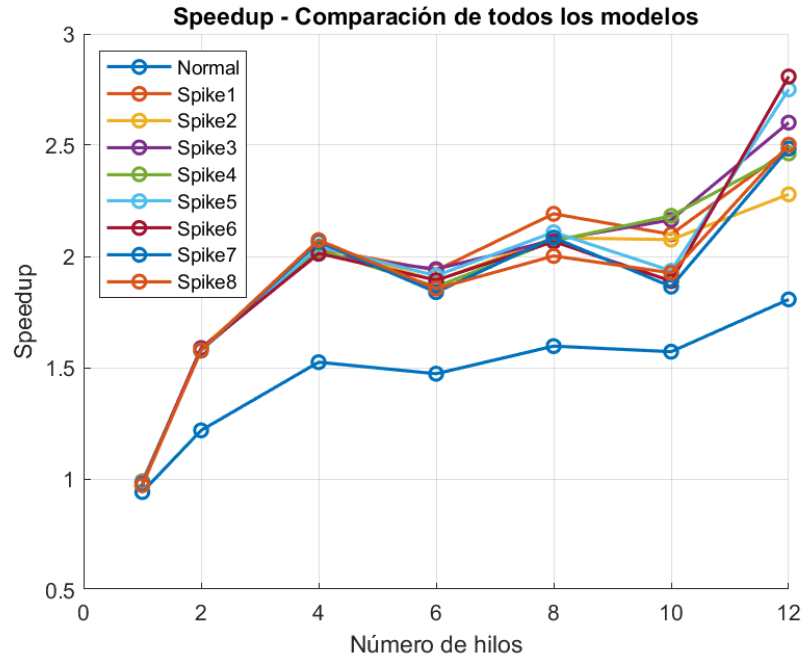


Figura 3.2: Curva de SpeedUp (aceleración) del Modelo 7.

Como se puede observar en las Figuras 3.1 y 3.2, el comportamiento del algoritmo muestra una tendencia de crecimiento no lineal.

Analizando la gráfica de **SpeedUp** (Figura 3.2), se aprecia que la aceleración aumenta conforme se añaden hilos de ejecución, pasando de un valor base de 1.0 a superar el 2.0 con 12 hilos. A pesar de contar con más recursos de hardware, el algoritmo alcanza un punto de saturación donde añadir más hilos aporta una mejora marginal.

Por su parte, la gráfica de **Paralelidad/Eficiencia** (Figura 3.1) corrobora esta tendencia, mostrando que la capacidad del algoritmo para explotar el paralelismo crece rápidamente al inicio pero tiende a estabilizarse en torno a un valor cercano a 1.95 en el extremo superior de la prueba.

3.3.2 Análisis de la precisión (Error Promedio)

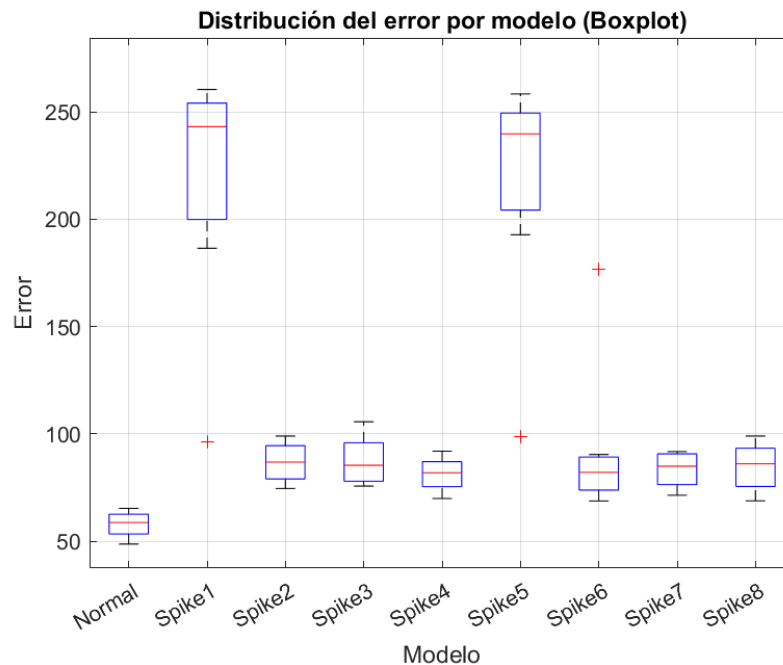


Figura 3.3: Comparativa del error promedio entre los distintos modelos neuronales.

La Figura 3.3 ofrece una comparativa crítica sobre la calidad de las soluciones encontradas.

- El **Modelo 1** presenta un error promedio menor, lo que sugiere que esta configuración (sin neurona) es la más adecuada para el problema planteado, logrando una convergencia más efectiva.
- Los **Modelos con Neurona** el error es muy dependiente del umbral es por eso que para la neurona 1 y 5 podemos decir que el umbral no era óptimo. Mientras que para la 2, 3, 4, 6, 7, 8 se observa que los resultados no distancian tanto del Modelo 1 aunque se quedan un poco cortos en cuanto a la aproximación a la solución óptima.

3.4. Conclusiones

A la luz de los resultados expuestos, puede afirmarse que la elección tanto del modelo neuronal como del umbral empleado resulta determinante en el comportamiento global del sistema.

Los experimentos ponen de manifiesto una notable disparidad en términos de precisión entre las distintas configuraciones analizadas. En este contexto, el Modelo 1 destaca por su robustez y estabilidad, logrando una convergencia consistente hacia soluciones de alta calidad. Por el contrario, el resto de variantes evaluadas presentan mayores dificultades para alcanzar niveles de rendimiento comparables.

Estos resultados subrayan la importancia de una selección cuidadosa del modelo y de sus hiperparámetros, ya que pequeñas variaciones pueden traducirse en diferencias significativas en la calidad de las soluciones obtenidas.

Bibliografía

- [1] Intel. Procesador intel® core™ i5-12450h. <https://www.intel.la/content/www/xl/es/products/sku/132222/intel-core-i512450h-processor-12m-cache-up-to-4-40-ghz/specifications.html>.
- [2] Eugene M. Izhikevich. Simple model of spiking neurons. *IEEE Transactions on Neural Networks*, 14(6):1569–1572, 2003.